# 할로우 나이트 보스는 어떻게 동작하는가

PlayMaker FSM·C# Action·오브젝트 참조를 분리해서 읽는 구조 조사 | 대표 사례: 호넷 1차전 계열

## 핵심 결론

보스별 State 클래스를 대량으로 상속하는 방식보다는, **FSM의 State에 재사용 Action을 배치**하여 판단·모션·이동을 조립한다. 호넷은 주 행동 FSM 외에도 경직·바늘·범위 감지 FSM을 연결한다. 단일 보스의 구성을 모든 보스의 공통 설계로 일반화하지 않는다.

### 01 FSM

State와 Event의 흐름
지상 돌진의 준비·실행·복구
주 FSM과 보조 FSM의 협력

### 02 클래스

PlayMakerFSM → Fsm → State
Action 계열과 실행 대상
상태 데이터와 C# 클래스 구분

### 03 할당·구현

Global Hero 참조 전달
자식·Owner·변수 연결
타이밍·중단·정리 정책

**문서 구성** 2 근거 범위 / 3 전체 FSM / 4 돌진 상세 / 5 FSM 간 협력 / 6 클래스 관계 / 7 Action 구현 / 8 오브젝트 할당 / 9 실행·수명 / 10 적용 시사점 / 11-12 출처

### 읽는 기준

공식 자료는 PlayMaker 사용 사실을, 비공식 공개 추출본은 구체적인 상태·Action·참조를 뒷받침한다. 원작 실행·공식 소스 열람·전체 DLL 분석은 수행하지 않았다.

근거 [A1] Team Cherry | Unity와 PlayMaker 도입 / [A2] Unity | Hollow Knight 제작 사례 / [B1] 추출본 | Hornet / Control

# 확인된 사실과 추정의 경계

A = 공식 자료 / B = 공개 추출본·직접 작성된 모딩 코드 / C = 설명용 재구성·프로젝트 적용 의견

| 구분 | 이번 조사에서 확인한 내용 | 단정하지 않는 내용 |
|---|---|---|
| A / 공식 | Team Cherry는 적 행동을 시각적 FSM으로 제작했다고 설명한다. Unity도 적·상호작용 요소의 PlayMaker 사용을 소개한다. | 모든 보스가 동일한 State·클래스 구조를 공유한다는 의미는 아니다. |
| B / 호넷 추출본 | Fungus1_04_boss / Hornet Boss 1 / Control. 시작 State는 Pause. 추출본의 State 수는 77개. | 빌드 번호는 미확인. 재대결·Godhome 도입 경로도 포함되어 순수 1차전 노드만의 수가 아니다. |
| B / 교차 확인 | Stun Control, 자식 Needle Control, A Dash Range FSM을 확인. 모딩 코드도 Control·Needle 접근을 사용한다. | 모드의 Hornet1Control은 모드가 추가한 클래스이며 원작 Controller가 아니다. |
| C / 다이어그램 | 관찰된 이름·연결을 사용하되 핵심 경로만 선별해 다시 배치한다. API 멤버도 일부만 표기한다. | 원작 에디터 배치, 전체 C# 상속 트리, 전체 Component 목록의 복제본이 아니다. |

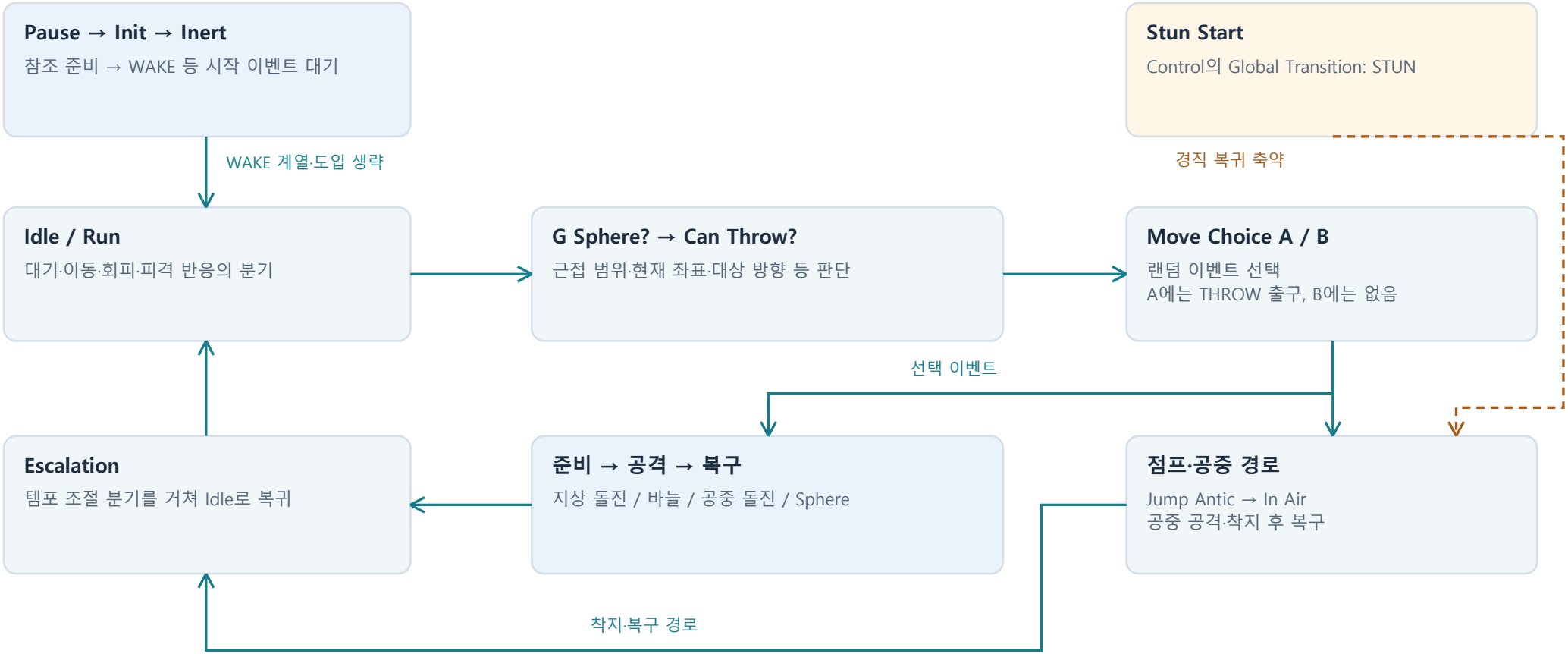
**해석 주의**

최신 PlayMaker 문서는 2의 API도 포함한다. 본 문서는 원작 분석에 필요한 1 계열 (FsmStateAction, OnEnter/OnUpdate/OnExit)을 사용한다. 최신 Sub-State·Region 기능을 원작에서 썼다고 가정하지 않는다.

근거 [A1] Team Cherry | Unity와 PlayMaker 도입 / [A2] Unity | Hollow Knight 제작 사례 / [B1] 추출본 | Hornet / Control / [P6] PlayMaker | 1/2 API 구분

# 주 행동 FSM: 판단과 실행이 같은 그래프에 있다

Control의 실제 상태명을 사용한 선별 흐름도. 여러 State를 묶은 상자는 읽기용 축약이며 HFSM 선언이 아니다.




경직 복귀는 실제로 **Stun Air → Stun Land → Stun Recover → Set Jump Only**로 이어진다. 주황 점선은 이 경로에서 점프·공중 경로로 이어지는 과정을 축약한 것이다.

근거 [B1] 추출본 | Hornet / Control / [B2] 추출본 | Stun Control

# 지상 돌진 하나를 State와 Action으로 분해하면

GDash Antic → G Dash → GDash Recover1 → GDash Recover2 → Escalation. 아래 내용은 활성 Action 기준이다.

**GDash Antic**
대상을 향해 정렬
속도 0
준비 모션·몸통 크기

→

**G Dash**
효과·Hit GDash 활성
수평 속도 적용
대시 모션

→

**GDash Recover1**
Hit GDash 비활성
감속
첫 복구 모션

→

**GDash Recover2**
감속 유지
몸통 크기 복원
두 번째 복구 모션

→

**Escalation**
체력·진행 조건 확인
대기 템포 설정
Idle로 복귀

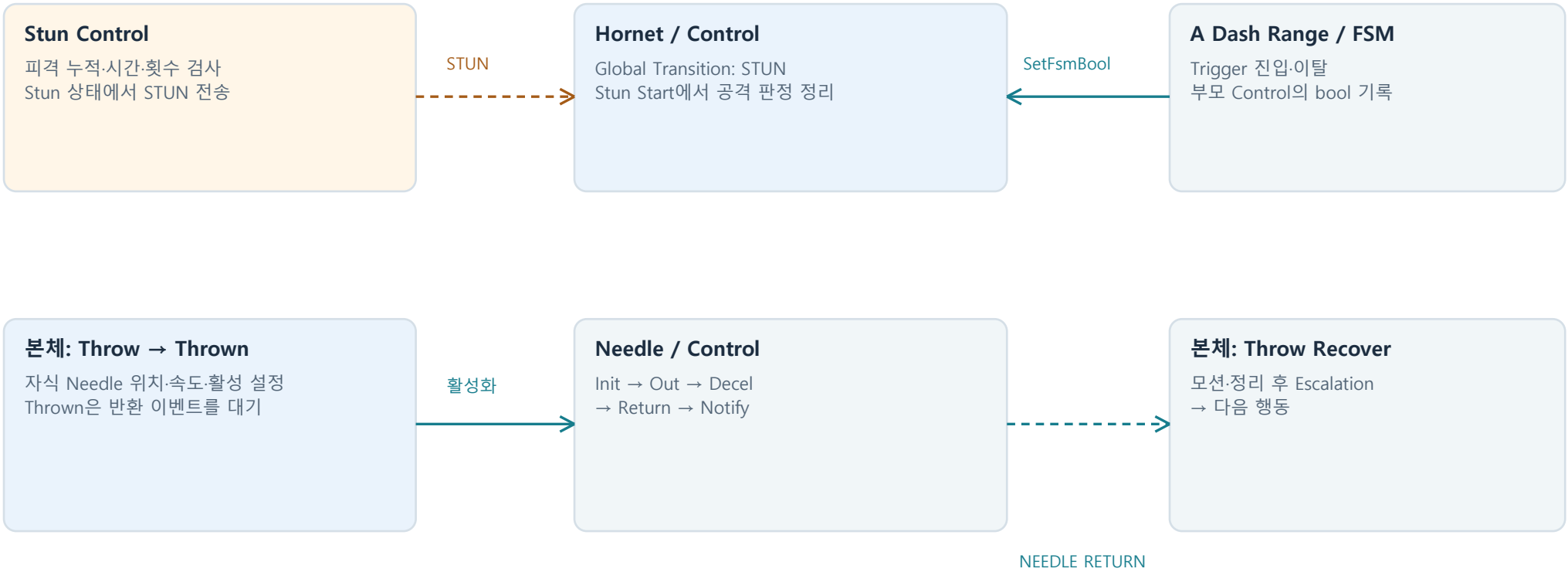
| 전이 계기 | 실제 관찰값 | 구현상 의미 |
|---|---|---|
| 준비 → 돌진 | Tk2dPlayAnimationWithEvents의 animationCompleteEvent = FINISHED | 모션 종료가 다음 State를 시작한다. |
| 돌진 → 복구 | Wait 0.35초 또는 좌/우 측면 충돌이 FINISHED 발행 | 고정 시간이든 벽 충돌이든 같은 복구 경로를 사용한다. |
| 복구 → 다음 행동 | 두 복구 모션의 완료 후 Escalation → Idle | 공격 판정 해제·감속·몸통 설정 복원을 그래프에 명시한다. |

**중요:** 상태가 바뀌면 모든 속도·Collider·활성 상태가 자동으로 복구되는 것은 아니다. 이 사례는 후속 상태와 경직 진입 상태가 정리를 수행한다. Action 자체의 취소 처리는 해당 Action 구현에 달려 있다.

근거 [B1] 추출본 | Hornet / Control / [P2] PlayMaker | Action / [P3] PlayMaker | Event

# 본체·경직·투사체·감지 범위가 협력한다

이벤트·변수 쓰기는 협력 방식이다. 여러 FSM이 존재한다는 사실만으로 HFSM이라고 부르지 않는다.




**참조 경로:** 본체는 Needle을 자식 참조로 보관한다. 바늘의 Notify는 **BroadcastAll**로 NEEDLE RETURN을 보낸다. 본체만 지정하는 직접 호출로 바꿔 그리지 않았다.

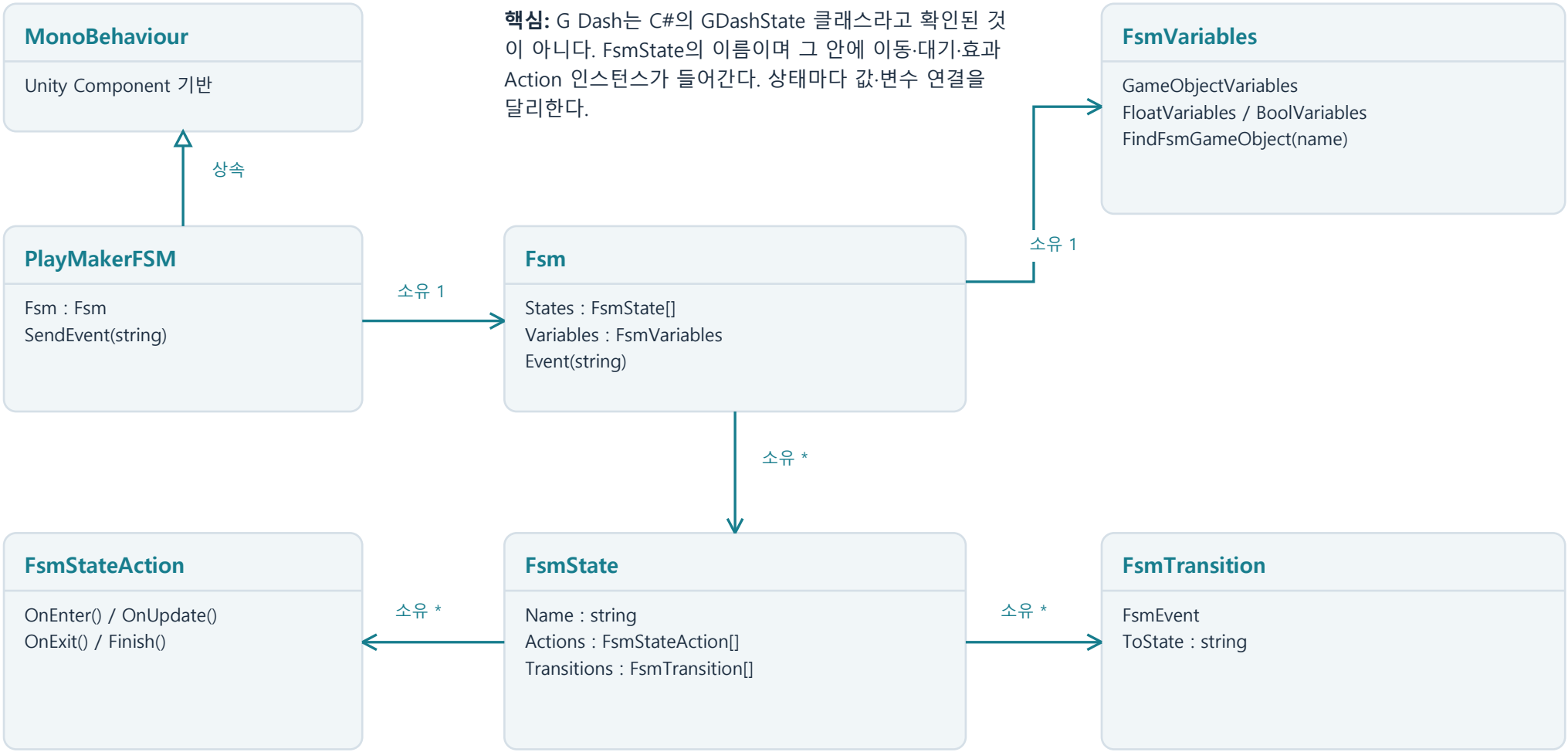
**용어 구분:** Control의 Global Transition은 해당 FSM에서 어느 상태든 STUN을 받을 수 있는 경로다. 이벤트의 Global 설정·Broadcast 전송 범위와는 별개의 개념이다.

근거 [B1] 추출본 | Hornet / Control / [B2] 추출본 | Stun Control / [B3] 추출본 | Needle / Control / [B4] 추출본 | A Dash Range

# C# 클래스 관계: State 데이터가 Action을 소유한다

PlayMaker 1 계열의 대표 API 관계를 축약. 원작 전체 클래스 덤프가 아니며 접근자·내부 필드는 생략한다.

**핵심:** G Dash는 C#의 GDashState 클래스라고 확인된 것이 아니다. FsmState의 이름이며 그 안에 이동·대기·효과 Action 인스턴스가 들어간다. 상태마다 값·변수 연결을 달리한다.

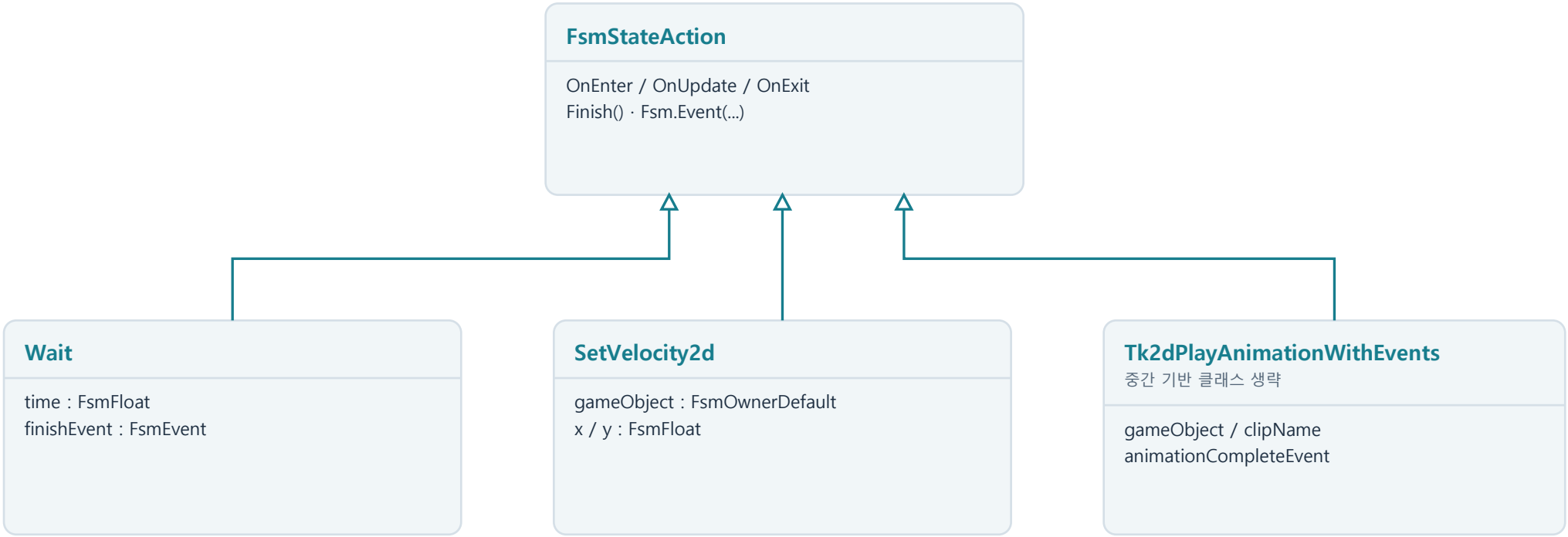



근거 [P1] PlayMaker | FSM Component / [P2] PlayMaker | Action / [P4] PlayMaker | 변수 / [P6] PlayMaker | 1/2 API 구분 / [B1] 추출본 | Hornet / Control

# 보스별 State 상속보다 작은 Action의 조립

C#은 재사용 실행 기능을, 그래프는 그 기능을 호출할 대상·설정·전이 이벤트를 구성한다.

## 원작에서 관찰되는 Action 계열




위 화살표는 FsmStateAction **계열**을 뜻하며 직접 부모 타입을 전부 확인한 상속 덤프가 아니다. Action 하나마다 GameObject에 Component를 추가하는 방식과 구분한다.

같은 Wait 클래스라도 상태별로 시간·완료 이벤트를 다르게 저장한다. 같은 이동 Action에는 Owner 또는 다른 GameObject, 상수 또는 FSM 변수를 할당할 수 있다.

근거 [B1] 추출본 | Hornet / Control / [P2] PlayMaker | Action / [P4] PlayMaker | 변수 / [P6] PlayMaker | 1/2 API 구분

# 오브젝트는 어디에서 가져와 어디에 넣는가

관찰된 경로 중심의 재구성. 모션·물리는 Action의 실행 대상으로 표시했으며 전체 Component 부착 목록은 아니다.

### 관찰된 오브젝트 구성

**Hornet Boss 1**
PlayMakerFSM: Control
PlayMakerFSM: Stun Control

**자식 오브젝트**
Needle / Control
Thread
Needle Tink (Pause에서 탐색)
A Dash Range / FSM
Hit GDash / Hit ADash
Sphere Ball
G Dash Effect 등

모션·물리 실행 대상:
tk2d 모션 / Rigidbody2D / Collider2D

| 할당 방식 | 호넷의 관찰 사례 |
|---|---|
| Owner | GetOwner로 Self 확보. OwnerDefault FSM Owner는 해당 Action이 속한 FSM의 GameObject를 뜻한다. |
| Player | Init에서 Global Hero → 로컬 Hero Obj. 방향·각도 판단 Action이 이를 읽는다. |
| 자식 | 변수에 Needle·효과·판정 경로가 저장됨. Pause의 FindChild로 Needle·Needle Tink도 확보한다. |
| 값·변수 | 대시 속도·몸통 크기는 변수, Wait 시간은 상수 또는 변수. Action 필드에 연결한다. |
| 실행 연결 | 범위 자식은 GetParent 후 부모에 bool 기록. Needle은 복귀 이벤트로 본체에 알린다. |

**Prefab 제약:** Prefab 에셋은 Scene 인스턴스를 직접 저장할 수 없다. 자기 계층·에셋 참조와 런타임 외부 참조 전달을 분리한다. Global Hero 경로는 그 한 사례다.

근거 [B1] 추출본 | Hornet / Control / [B3] 추출본 | Needle / Control / [B4] 추출본 | A Dash Range / [P1] PlayMaker | FSM Component / [P5] PlayMaker | Scene 참조

# 실제로 구현할 때의 작업 단위와 실행 수명

왼쪽은 관찰 구조를 재현하는 제작 절차, 오른쪽은 원작과 무관한 PlayMaker 1 계열 설명용 코드다.

| 작업 | 설정·코드의 위치 |
| --- | --- |
| 1. 구성 | 보스·물리·모션·판정·효과를 구성하고 FSM Component를 부착한다. |
| 2. 연결 | FSM 변수와 Action 필드에 Owner·Player·자식·속도·모션명을 지정한다. |
| 3. 전이 | 준비·실행·복구에 Action을 배치하고 완료·충돌·경직 이벤트의 목적지를 정한다. |
| 4. 확장 | 기존 Action으로 부족할 때만 C# Action을 추가한다. 상태마다 클래스가 필요하지 않다. |
| 5. 정리 | 이동·중력·판정·자식 활성 상태를 정상 종료와 경직/사망 경로에서 복구한다. |

**해석 주의**

Action은 한 번 또는 지속 실행된다. 리스트에 있다고 작업 완료를 기다리는 순차 실행은 아니다. 상태가 끝나도 속도·판정 활성 상태는 자동 복원되지 않는다.

**설명용: 대상 X좌표를 읽는 Action**

```
public class ReadTargetX : FsmStateAction
{
    public FsmGameObject target;
    public FsmFloat result;

    public override void OnEnter()
    {
        result.Value =
            target.Value.transform.position.x;
        Finish();
    }
}
```

target을 Hero Obj에, result를 로컬 float 변수에 연결한다. Finish()는 이 Action의 완료이며 임의의 전이 이벤트와 동일한 개념이 아니다. 명시적 이벤트가 필요하면 Fsm.Event(...)를 사용한다.

근거 [P2] PlayMaker | Action / [P3] PlayMaker | Event / [P4] PlayMaker | 변수 / [P6] PlayMaker | 1/2 API 구분 / [B1] 추출본 | Hornet / Control

# 배워야 할 것은 에셋보다 책임과 연결 방식

여기부터는 ProjectRE 적용에 대한 해석. 원작이 같은 SO·Handler 구조를 사용했다는 의미가 아니다.

| 관찰한 방식 | 현재 프로젝트에서의 대응·판단 |
|---|---|
| FSM에 판단·행동 순서 구성 | AgentStateBase + ITransitionRule의 명시적 전이를 유지하면서 패턴 선택·공격 수명을 설계할 수 있다. PlayMaker 도입이 필수는 아니다. |
| 작은 실행 기능 재사용 | 타격·이동·투사체 기능을 재사용하되, 보스마다 같은 입력 계약이나 거대한 공통 기반 클래스를 만들 필요는 없다. |
| 모션·타이머·충돌이 진행 이벤트 생성 | 공격 시작·종료·중단의 결정자를 명시한다. Animator Bool만으로 전투 상태를 추정하지 않는다. |
| 보조 FSM의 별도 책임 | 본체와 독립 수명인 기능은 분리할 수 있다. 다만 FSM 사이의 실행 충돌·소유권과 통신 비용도 생긴다. |
| 개체별 변수·외부 참조 | 현재 MonsterAttackHandler의 실행 SO 복제와 비교할 수 있으나, 원작이 SO 복제 전략을 썼다는 증거는 없다. |

### 이번 조사로 확정하지 않는 사항

모든 보스의 같은 기반 Controller / 정확한 패턴 확률·반복 제한 / 체력·사망 C# 전체 호출 체인 / 모든 피격이 공격을 중단하는지 / 매 프레임 Player 좌표를 읽는지 / 모든 보스가 Raycast를 쓰지 않는지.
**확인된 호넷 사례에는 Trigger 범위와 좌표 경계 비교가 함께 있다.**

근거 [A1] Team Cherry | Unity와 PlayMaker 도입 / [B1] 추출본 | Hornet / Control / [B4] 추출본 | A Dash Range

# 공식 자료: 사용 사실과 도구의 동작 원리

제목은 클릭 가능한 원문 링크. 근거 ID는 본문 하단과 대응한다. 확인일: 2026-10-19

**[A1] Team Cherry | Unity와 PlayMaker 도입**

개발자가 적 AI를 FSM으로 구성한 이유와 작업 방식을 설명한다.

**[A2] Unity | Hollow Knight 제작 사례**

적·상호작용 요소의 PlayMaker 사용과 일부 사용자 스크립트 병용을 확인한다.

**[P1] PlayMaker | FSM Component**

GameObject 부착 방식과 Prefab 내부 참조·Scene 참조의 제약을 설명한다.

**[P2] PlayMaker | Action**

상태 안의 Action과 실행 개념을 확인한다. 최신 기능은 원작에 소급 적용하지 않았다.

**[P3] PlayMaker | Event**

Event와 Transition의 구분. 전송 범위와 FSM 내부 전이도 따로 읽어야 한다.

**[P4] PlayMaker | 변수**

Action 필드의 상수·변수 연결, GameObject 변수와 Global 값의 의미를 설명한다.

**[P5] PlayMaker | Scene 참조**

Prefab에서 외부 Scene 오브젝트를 실행 중 확보·전달하는 방법을 설명한다.

**[P6] PlayMaker | 1/2 API 구분**

1의 FsmStateAction/OnEnter/OnUpdate/OnExit와 2의 API가 다름을 확인한다.

# 공개 관찰 자료와 추가 검증 범위

비공식 추출본은 원작의 공개 설계서가 아니다. 수정된 모딩 구현과 원작 구성을 섞지 않는다.

### [B1] 추출본 | Hornet / Control

시작 상태·77개 상태·전이·Action 활성 여부·변수·참조를 조사. 도식은 핵심 경로만 선별했다.

### [B2] 추출본 | Stun Control

같은 Hornet 오브젝트의 경직 판단 FSM과 STUN 송신을 확인했다.

### [B3] 추출본 | Needle / Control

바늘의 이동·복귀·NEEDLE RETURN Broadcast와 HORNET KILLED → Destroy 경로를 확인했다.

### [B4] 추출본 | A Dash Range

Trigger 진입/이탈로 부모의 A Dash Range bool을 기록하는 구성을 확인했다.

### [B5] 모딩 코드 | DoubleBosses

모드의 Hornet1Control이 Control·Needle에 접근하는 부분만 교차 확인. 대상은 Godhome 장면이다.

**추출 기준** windplusflower/hk-api-skill
커밋 fdac21584342cdaf04f521aa23b36ac71f7a6eea
Fungus/Fungus1_04_boss.unity, 원본 자원 level133. 게임 빌드 번호는 미확인.

**추가 확인** 합법적으로 보유한 동일 버전 게임에서 Control·Stun Control·Needle을 읽기 전용으로 관찰하고, Player 참조·활성 State·Event·판정 오브젝트를 함께 기록한다. 이번 작업은 게임 실행·모드 설치·프로젝트 코드 변경을 하지 않았다.